SHIPSHAPE AUDIT DEMO

Audit, Improvement, and Deployment Summary

A focused walkthrough of how the system works, what was audited, what improved, and what was validated in a live environment.

AUDIT SCOPE
7 Required Categories

VALIDATION STYLE
Baseline → Fix → Proof

LIVE DEPLOY
AWS + Public URL

Talk Track

1. Mental model of the system
2. Audit approach and evidence trail
3. Key architectural discoveries
4. AI cost analysis and telemetry validation
5. Live deployment and final conclusion

SYSTEM MODEL

Start With the Architecture

Before changing code, the first step was to understand the operating model of the application and the architectural decisions that drive the product.

`web`

- React frontend
- single-page application
- route-driven UX

→

`api`

- Express backend
- document and workflow logic
- AI analysis endpoints

→

`shared`

- shared contracts
- cross-package types
- product-level consistency

CORE MODEL

Issues, projects, docs, and weekly artifacts all live in one unified document system.

REALTIME

Collaboration is powered by Yjs and WebSockets.

PRODUCT COHERENCE

Typed associations are the backbone of flexibility across features.

WHY IT MATTERS

The audit focused on improving the architecture already present, not replacing it.

AUDIT METHOD

Seven Categories, One Evidence Standard

Required Categories

- Type safety
- Bundle size
- API response time
- Database query efficiency
- Test coverage and quality
- Runtime error and edge-case handling
- Accessibility compliance

Working Method

- capture a baseline
- identify the root cause
- implement a fix
- record the after-state
- preserve reproducible proof

GOAL

Improve the system in measurable ways, not just patch isolated defects.

OUTPUT

A clear engineering trail instead of unsupported claims.

REVIEW STYLE

Before-and-after evidence for every category.

RESULT

The audit is defensible, repeatable, and easy to review.

KEY DISCOVERIES

Three Important Findings From the Codebase

TESTING INFRASTRUCTURE

Playwright uses isolated per-worker environments, including separate database and server resources. That is a strong pattern for reliable end-to-end testing.

ARCHITECTURAL CENTER

The unified document model with typed associations is the core design decision that gives ShipShape flexibility and product coherence.

ACCESSIBILITY PATTERN

The frontend already includes route-level focus management, which is a meaningful accessibility practice for a single-page application.

These discoveries mattered because they changed the audit posture from “replace weak systems” to “strengthen the strong core.”

AI COST ANALYSIS

Validated Telemetry, Not Assumptions

What Was Validated

- successful Braintrust traces
- prompt, completion, and total tokens
- estimated cost
- latency and request metadata

Confirmed Example

- provider: `OpenAI`
- model: `gpt-4.1-mini`
- `2,009` total tokens
- `$0.0022136` per request

MONTHLY USERS	ASSUMPTION	ESTIMATED MONTHLY COST
100	4 analyses per user / month	$0.8854
1,000	4 analyses per user / month	$8.8544
10,000	4 analyses per user / month	$88.5440
100,000	4 analyses per user / month	$885.4400

Application-side AI cost is validated. Coding-agent cost still requires a separate ledger.

DEPLOYMENT

Live Validation in a Public Environment

Deployed Environment

AWS

Public URL:

`https://d1woqw06xb054i.cloudfront.net`

Why This Environment

- publicly accessible for review
- matched the full-stack submission requirement
- safe place to validate the improved build end to end

Frontend validated through the public CloudFront URL.

Backend validated in the AWS deployment path used for the improved application.

Telemetry validated against successful live requests instead of local-only assumptions.

CONCLUSION

The Architecture Was Strong. The Work Was About Maturity.

ShipShape already has a strong architectural core. The real opportunity was to make that architecture more consistent, more testable, more accessible, and more operationally mature.

Not a rewrite.
The system did not need reinvention; it needed disciplined strengthening.

Measured improvements.
Each category was handled with baseline evidence, root-cause analysis, implementation, and proof.

End-to-end validation.
The result was not only code changes, but a validated, deployed, and observable system.

Close with questions or a short live walk-through in the app.